

AXIOM Agent (constitutional dev agent)

A single-loop LLM caller that runs four task modes against a chosen OpenAI- or Anthropic-compatible backend, with the constitutional system prompt loaded from

`axiom_files/core/axiom_agent.axiom` and every interaction signed into `axiom_agent_memory.jsonl`.

This document is the operator-facing reference. The implementation lives in `axiom_agent.py:1`. For the multi-loop sandboxed variant, see `AUTONOMOUS_AGENT.md`. For the four-layer reviewer variant, see `DEV_AGENT.md`.

Modes

Mode	Purpose
<code>feature</code>	Spec → tests → implementation for a new capability
<code>bug_hunt</code>	Scan target files, rank by severity, propose fixes
<code>efficiency</code>	Profile a pipeline, measure baseline, optimize
<code>reasoning_lab</code>	Propose an experiment with the ISOLATION flag set

Each task is matched against the known-bug catalogue (`BUG_PATTERNS` in `axiom_agent.py`) and any matching bug IDs are injected into the prompt as `RELEVANT BUG PATTERNS`.

Backend selection

`AxiomAgent(backend="auto" | "openai" | "anthropic")` resolves in this order:

1. OpenAI-compatible: `OPENAI_API_KEY` (or `NVIDIA_API_KEY`) with optional `AXIOM_BASE_URL` / `AXIOM_MODEL`. Default model `qwen/qwen3-235b-a22b`.
2. Anthropic: `ANTHROPIC_API_KEY`. Default model `claude-sonnet-4-6`.
3. Neither set → agent returns `status: "offline"` with the bug-pattern matches but no LLM response.

CLI

```
python axiom_agent.py --task "write a guard for X" --mode feature
python axiom_agent.py --task "scan for BUG-001" --mode bug_hunt
python axiom_agent.py --task "profile guard pipe" --mode efficiency
python axiom_agent.py --task "test new branch" --mode reasoning_lab
```

```
python axiom_agent.py --profile      # show pipeline profile
python axiom_agent.py --bugs        # list known bug patterns
```

API

```
from axiom_agent import AxiomAgent

agent = AxiomAgent()                # backend auto-detect
out = agent.run_task("write a guard for X", mode="feature")
# out: { status, mode, task, response, relevant_bugs, model, backend }
```

Memory and signing

Every completed task appends a JSONL entry to `axiom_agent_memory.jsonl`, HMAC-SHA256 signed via `derive_key(b"axiom-agent-memory-v1")`. The file is append-only and the agent does not read its own memory at run time — it is an audit artifact, not a context source.

Trust constraints

The agent has no sandbox, no autonomous tool dispatch, and no ability to mutate constitutional fields. It is a single chat completion per call, gated only by the system-prompt contract. For sandboxed multi-step execution use `axiom_autonomous_agent.py`; for code-review governance use `axiom_dev_agent_v2.py`.